Linguaggi e Compilatori

Appunti del Corso

Alessia Cassetta

19 marzo 2026

Indice

1 Fondamenti e Architettura dei Compilatori **1**
1.1 Introduzione ai Linguaggi . . . 1
1.1.1 Linguaggi Compilati . . . 1
1.1.2 Linguaggi Interpretati . . . 1
1.1.3 Approcci Ibridi (Compilazione + Interpretazione) . . . 1
1.2 Linguaggi e Macchine Astratte . . . 2
1.3 Architettura del Compilatore . . . 2
1.3.1 Front-end (Fase di Analisi) . . . 2
1.3.2 Back-end (Fase di Sintesi) . . . 3
1.4 Scoping e Gestione della Memoria . . . 4
1.4.1 Scoping Statico vs Dinamico . . . 4
1.4.2 Environments e States . . . 4
1.4.3 Passaggio Parametri . . . 5
1.5 Introduzione ai Compilatori Giocattolo . . . 6

2 Analisi Lessicale e Automi a Stati Finiti **7**
2.1 Analisi Lessicale (Lexical Analysis) . . . 7
2.1.1 Il Ruolo del Lexer . . . 7
2.1.2 Categorie Lessicali e Symbol Table . . . 8
2.1.3 Caratteri di Lookahead . . . 8
2.2 Stringhe, Linguaggi ed Espressioni Regolari . . . 9
2.2.1 Terminologia per le Stringhe . . . 9
2.2.2 Linguaggi di Stringhe e Operatori . . . 9
2.2.3 Espressioni Regolari (ER) . . . 9
2.2.4 Definizioni Regolari ed Estensioni . . . 10
2.3 Automi a Stati Finiti . . . 11
2.3.1 Rappresentazione e Accettazione NFA . . . 11
2.3.2 Differenze Fondamentali tra DFA e NFA . . . 12
2.4 Convertire un NFA in un DFA . . . 13
2.4.1 Alcune definizioni . . . 13
2.4.2 Subset Construction: idea di base . . . 13
2.4.3 Il DFA finale . . . 14
2.4.4 Esempio di Subset Construction . . . 14
2.4.5 Complessità del Subset Construction . . . 15
2.5 Simulare un NFA . . . 15
2.6 Da ER a NFA: Algoritmo di McNaughton, Yamada e Thompson . . . 15
2.6.1 Esempio . . . 17
2.7 Programmi che usano ER: Confronto Costi . . . 18
2.8 Lexical Analysis: dagli Automi al Lexer . . . 19
2.8.1 Struttura di un lexer (generato) . . . 19
2.8.2 Riconoscimento vs Lexing . . . 19
2.8.3 Specifiche per un generatore: ER multiple . . . 19

2.8.4 Conflitti inter-ER e intra-ER 20
2.8.5 Lexer basato su NFA 20
2.8.6 Lexer con NFA ⇒ DFA 21
2.8.7 L'operatore "Lookahead" 22
2.8.8 Minimizzazione degli stati di un DFA 22
2.9 Generazione Automatica di Lexer: Lex 23

3 Analisi Sintattica e Parsing 25
3.1 Sintassi e Grammatiche Context-Free 25
3.1.1 Convenzioni Notazionali 25
3.2 Gerarchia e Tipi di Grammatiche 26
3.3 Derivazioni e Linguaggio 26
3.4 Parse Tree e Parsing 28
3.5 Ambiguità, Associatività e Precedenza 29
3.6 Potenza Espressiva: Regular vs Context-Free 30
3.7 Ristrutturazione delle Grammatiche 30
3.7.1 Eliminazione dell'Ambiguità: Dangling Else 31
3.7.2 Eliminazione della Ricorsione a Sinistra 31
3.7.3 Fattorizzazione a Sinistra 32
3.8 Algoritmi di Parsing: Bottom-Up e Top-Down 32
3.8.1 Predictive Parsing e Recursive-Descent 33
3.9 Top-Down Parsing: First, Follow e Tabelle LL(1) 33
3.9.1 Grammatiche LL(1) 35
3.9.2 Predictive Parsing e Costruzione della Tabella M 35
3.9.3 Meccanica del Predictive Parsing senza ricorsione 35

4 Traduzione Semantica e Rappresentazioni Intermedie 37
4.1 Traduzione Guidata dalla Sintassi 37
4.1.1 Syntax-Directed Semantic Definitions (Approccio Dichiarativo) . 37
4.1.2 Syntax-Directed Translation Scheme (Approccio Imperativo) . . 37
4.2 Abstract Syntax Tree 37
4.3 Symbol Table per la Gestione dello Scope 38
4.4 Generazione di Rappresentazioni Intermedie (IR) 38
4.4.1 Traduzione dall'AST al 3-Address Code: l-value e r-value 39

1

Fondamenti e Architettura dei Compilatori

1.1 Introduzione ai Linguaggi

1.1.1 Linguaggi Compilati

Un linguaggio compilato, come il C o il C++, viene interamente tradotto in codice macchina prima di essere eseguito. Questo processo, chiamato *Ahead-Of-Time* (AOT) compilation, permette al programma di essere eseguito molto velocemente, poiché il processore legge istruzioni native. Le fasi principali sono:

1. **Preprocessamento:** Gestisce l'inclusione di file header e la sostituzione di macro.
2. **Compilazione:** Traduce il codice preprocessato in linguaggio assembly o direttamente in codice oggetto (rappresentazione intermedia).
3. **Linking (Collegamento):** Il linker combina uno o più file oggetto con le librerie esterne necessarie, generando infine un file **eseguibile** (es. `.exe` o un binario ELF).

1.1.2 Linguaggi Interpretati

Linguaggi come Python, JavaScript o Lisp classico vengono eseguiti da un programma chiamato **interprete**. Non c'è la generazione di un file eseguibile preventivo. L'interprete legge, analizza ed esegue il codice sorgente riga per riga (o blocco per blocco) a runtime. Questo offre massima flessibilità, facilità di debugging e portabilità, ma rende l'esecuzione tendenzialmente più lenta rispetto al codice compilato, a causa dell'overhead di traduzione durante l'esecuzione.

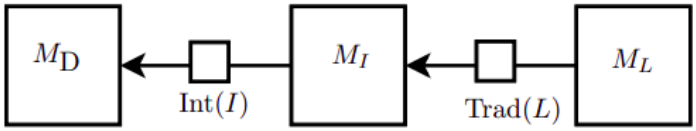
1.1.3 Approcci Ibridi (Compilazione + Interpretazione)

Oggi, il 99% dei linguaggi moderni (come Java e C#) utilizza un approccio ibrido:

- Il codice sorgente viene prima **compilato** in un linguaggio intermedio indipendente dall'architettura (es. *Bytecode*).
- A runtime, una Macchina Virtuale (es. la JVM) **interpreta** questo bytecode oppure lo compila *Just-In-Time* (JIT) in codice macchina nativo per massimizzare le prestazioni.

1.2 Linguaggi e Macchine Astratte

Per comprendere come viene eseguito un programma, è utile introdurre il concetto di *macchina astratta*.



Assiomi per ogni linguaggio L

- Esiste una **macchina** M_L in grado di "capire" ed eseguire direttamente programmi scritti in L.
- Per ragioni pratiche, esiste spesso un **linguaggio intermedio** L_I e una relativa macchina M_I che lo accetta.
- Esiste un **Traduttore** $Trad(L)$ (o compilatore) che converte i programmi da L a L_I.
- Esiste un **Interprete** "puro" $Int(L)$ che esegue direttamente i programmi. E quindi traduce da I a D.

In questo contesto, M_L e M_I fungono da **parser** per i rispettivi linguaggi. Con l'evoluzione dell'informatica, la macchina base M_D è diventata il **Sistema Operativo** in combinazione con l'hardware: il linguaggio D è il **linguaggio macchina** del processore arricchito dalle *System Calls* (chiamate di sistema). In pratica, $\langle M_L, Trad(L)\rangle$ spesso convivono in un unico sistema, ma teoricamente distinguerli ci aiuta a capire la separazione tra l'analisi del testo (parsing) e la sua trasformazione o esecuzione.

1.3 Architettura del Compilatore

Il processo di compilazione è tradizionalmente diviso in due macro-fasi: il **Front-end** e il **Back-end**. Tutte le fasi comunicano con una struttura dati centrale chiamata **Symbol Table** (Tabella dei Simboli).

1.3.1 Front-end (Fase di Analisi)

Si occupa di analizzare il codice sorgente, verificarne la correttezza e comprenderne la struttura logica. È dipendente dal linguaggio sorgente ma indipendente dalla macchina su cui girerà il programma.

- **Lexical Analyzer (Scanner):** Scompone il testo in unità logiche (Token).
- **Syntax Analyzer (Parser):** Verifica la grammatica e costruisce l'albero sintattico.
- **Semantic Analyzer:** Controlla il significato logico (es. type checking).
- **Intermediate Code Generator:** Produce una rappresentazione intermedia (IR) standardizzata.

Lexical Analysis (Scanner) Il Lexer legge il flusso continuo di caratteri e raggruppa le sequenze (chiamate *lexeme*) in categorie lessicali. Il risultato è un **token**, espresso come coppia `<nome, valore>`.

Esempio Pratico Consideriamo l'istruzione: `position = initial + rate * 60`

Lexeme	Categoria	Token generato	Azione in Symbol Table
`position`	Identificatore	`<id, 1>`	Inserito/Cercato alla pos. 1
`=`	Assegnamento	`<assign, - >`	Nessuna (è un operatore fisso)
`initial`	Identificatore	`<id, 2>`	Inserito/Cercato alla pos. 2
`+`	Operatore Arit.	`<arop, + >`	Nessuna
`rate`	Identificatore	`<id, 3>`	Inserito/Cercato alla pos. 3
`*`	Operatore Arit.	`<arop, * >`	Nessuna
`60`	Costante Numerica	`<numconst, 60>`	Inserito valore

Il Lexer si occupa anche di eliminare gli spazi bianchi e i commenti, alleggerendo il lavoro del Parser.

Syntax Analysis (Parser) Il Parser riceve i token e verifica che rispettino le **Grammatiche Context-Free (CFG)** del linguaggio (tipo 2 nella gerarchia di Chomsky).

- Costruisce un **Parse Tree** (Albero Sintattico), che rappresenta la struttura gerarchica dell'istruzione tenendo conto della precedenza degli operatori (esempio: la moltiplicazione prima dell'addizione).
- Se la sequenza di token viola le regole grammaticali, lancia un *Syntax Error*.

Semantic Analysis Verifica la coerenza logica che non può essere catturata da una grammatica context-free. È quindi un insieme di controlli $\{C_1, \ldots, C_n\}$ t.c. $\forall C_i$, C_i può essere eseguito in tempo di compilazione. C_i non può essere eseguito dal Parser. Esempi classici:

- **Type Checking:** Assicurarsi che non si sommi un intero con una stringa (se il linguaggio non lo permette).
- **Verifica dei Parametri:** Controllare che una chiamata a funzione abbia il numero e il tipo corretto di argomenti.
- **Dichiarazione variabili:** Verificare che una variabile sia stata dichiarata prima del suo utilizzo.

1.3.2 Back-end (Fase di Sintesi)

Si occupa di prendere la rappresentazione intermedia e generare il codice per la macchina target. È dipendente dall'hardware ma indipendente dal linguaggio sorgente.

- **Code Optimizer:** Migliora il codice intermedio per renderlo più veloce o fargli occupare meno memoria.
- **Code Generator:** Converte il codice intermedio ottimizzato nel linguaggio macchina/assembly target.

Note Importanti sulla Struttura

- **Interazione Lexer-Parser:** Spesso il Lexer non processa tutto il file in una volta, ma fornisce token *on-demand* al Parser (back and forth).
- **Symbol Table:** Ogni fase legge e aggiorna questa tabella, che contiene informazioni su variabili, tipi, funzioni e scope.
- **Generazione Automatica:** Strumenti come *Lex/Flex* e *Yacc/Bison* permettono di generare automaticamente Lexer e Parser a partire da espressioni regolari e grammatiche formali.

1.4 Scoping e Gestione della Memoria

La gestione dello scope (ambito di visibilità) determina in quale porzione di codice è valido e visibile un certo identificatore (variabile o funzione). Esistono due approcci principali per risolvere i riferimenti non locali: **Statico** e **Dinamico**.

1.4.1 Scoping Statico vs Dinamico

Scoping Statico (o Lessicale): Nello scoping statico, il binding di una variabile viene risolto analizzando la struttura testuale (lessicale) del codice sorgente. È il metodo utilizzato dalla stragrande maggioranza dei linguaggi moderni (C, C++, Java, Python, JavaScript).

- **Come funziona:** Se una variabile non è definita nello scope locale, il compilatore cerca nello scope del blocco che lo contiene (nel testo del sorgente), proseguendo verso l'esterno fino ad arrivare allo scope globale.
- **Vantaggi:** Il binding è determinabile a tempo di compilazione. Rende il codice molto più facile da leggere e permette forti ottimizzazioni da parte del compilatore.

Scoping Dinamico: Nello scoping dinamico, il binding viene risolto in base alla sequenza di chiamate a runtime (lo stack di esecuzione), indipendentemente da dove la funzione è scritta nel testo. Utilizzato oggi principalmente in alcuni script di shell (Bash) e vecchie versioni di Lisp.

- **Come funziona:** Se una variabile non è locale, l'interprete risale la catena delle chiamate a funzione (call stack) e usa la prima variabile con quel nome che incontra in memoria attiva in quel momento.
- **Svantaggi:** Comportamento imprevedibile: la stessa funzione può restituire risultati diversi a seconda di chi l'ha chiamata. Difficile da debuggare.

1.4.2 Environments e States

Un linguaggio di programmazione gestisce la memoria e i riferimenti su due livelli principali:

- **Environments (Ambienti):** Definiscono il mapping da nomi (identificatori) a locazioni, costanti o funzioni/procedure. Possono essere gestiti in modo statico o dinamico, a seconda delle regole di scoping del linguaggio.

- **States (Stati):** Definiscono il mapping da locazioni a valori. Questo aspetto è tipicamente dinamico.

Approfondimento: Scoping Statico e Dinamico Dato un linguaggio con dichiarazioni di variabili e un'occorrenza (uso) di una variabile x, lo scope definisce a quale dichiarazione si riferisce quell'uso di x all'interno del codice. La politica di scoping definisce il binding (dinamico o statico) dell'environment.

La **politica statica** è decidibile a tempo di compilazione.

- Data un'occorrenza di una variabile x, è possibile stabilire la sua dichiarazione D ispezionando il codice sorgente.
- È il tipo più comune fra i linguaggi moderni, poiché è più semplice da usare e non introduce errori a tempo di esecuzione.
- Si basa spesso sulla **struttura a blocchi** (zone di codice con delimitatori, ad esempio `{ }` oppure `begin`/`end`, che possono essere annidati).
- Lo scope di una dichiarazione D per x in un blocco B è tutto B, meno ogni sottoblocco B' di B in cui esiste una nuova dichiarazione D' per x.

La **politica dinamica** è decidibile solo a tempo di esecuzione.

- Data un'occorrenza di una variabile x, la sua dichiarazione D di appartenenza varia a run-time.
- Si trova solitamente in alcuni linguaggi di shell scripting e in sistemi di macro preprocessing.
- Rappresenta una frequente sorgente di errori a tempo di esecuzione (ad esempio, se l'esecuzione termina con un errore di tipo a causa di un binding inaspettato).

Scoping nei Linguaggi Object-Oriented I linguaggi orientati agli oggetti (OOL) presentano combinazioni di entrambe le politiche:

- **Access Control (Statico):** Metodi e campi di una classe possono essere dichiarati `public`, `private`, `protected`. Questo limita la visibilità ed è risolvibile in maniera statica.
- **Polimorfismo (Dinamico):** Se una classe D (sottoclasse di C) ridefinisce un metodo $m()$, e abbiamo un puntatore p di tipo C, la versione di $m()$ da eseguire tramite $p \to m()$ è nota solo a run-time.

In generale, introdurre una feature dinamica complica il runtime support e aggiunge possibili errori, ma ha senso se aumenta l'espressività in modo significativo (come nel caso del polimorfismo).

1.4.3 Passaggio Parametri

Esistono diverse modalità per passare parametri a una funzione f:

- **Call By Value (Per Valore):** L'espressione passata viene valutata e il risultato copiato in una locazione apposita (lo storage per la variabile locale in f). È

un approccio sicuro (la funzione non può cambiare il valore delle variabili del chiamante), ma inefficiente per strutture grandi, con costo $O(|S|)$. È usato in tutti i maggiori linguaggi, che offrono però metodi per aggirare la regola e aumentare le prestazioni.

- **Call By Reference (Per Riferimento):** Viene passata solo la locazione della variabile, usata direttamente come storage per la variabile locale in f. È molto efficiente (costo $O(1)$) ma introduce effetti collaterali, come l'accesso a variabili non locali, aliasing e dangling reference.
- **Call By Name (Per Nome):** L'espressione non viene valutata al momento del passaggio. Viene "legata" alla corrispondente variabile locale e valutata se e quando verrà usata all'interno di f. È potenzialmente molto efficiente, ma di difficile implementazione, presente perlopiù in linguaggi funzionali e interpretati.

1.5 Introduzione ai Compilatori Giocattolo

L'obiettivo di questa sezione è la costruzione (a livello logico e astratto) di due compilatori "giocattolo":

1. Un traduttore capace di convertire semplici espressioni aritmetiche dalla notazione infissa a quella postfissa.
2. Un compilatore per un mini-linguaggio imperativo.

Questo ci permetterà di osservare in anteprima, ad alto livello, alcune delle tecniche che verranno approfondite nelle parti successive del corso. Il piano d'azione si articola nei seguenti punti:

1. Sintassi, grammatiche e Parse Tree.
2. Traduzione guidata dalla sintassi (Syntax-Directed Translation).
3. Parsing Top-down.
4. Sviluppo del traduttore di espressioni.
5. Cenni di Analisi Lessicale (preludio alle sezioni future).
6. Gestione della Symbol Table per lo scope statico del mini-linguaggio.
7. Generazione di rappresentazioni intermedie.

2

Analisi Lessicale e Automi a Stati Finiti

2.1 Analisi Lessicale (Lexical Analysis)

2.1.1 Il Ruolo del Lexer

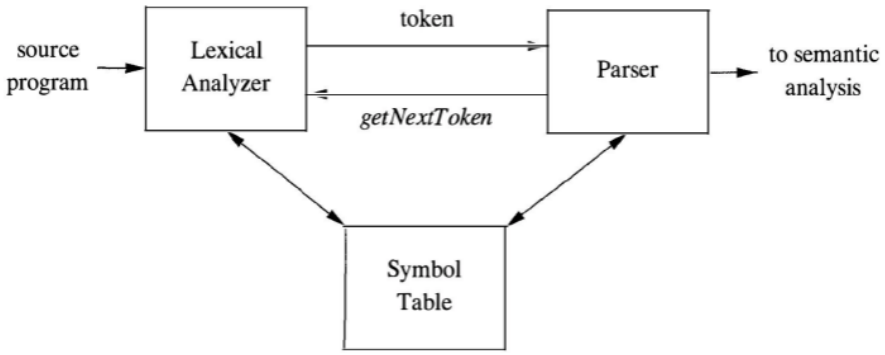


Figura 1: Interazione tra Lexer e Parser

L'Analisi Lessicale è la prima fase del processo di compilazione. Su richiesta del parser, il Lexer esegue i seguenti step:

1. Cerca un nuovo *lexeme* nell'input (a partire dalla posizione corrente).
2. Copia il lexeme in una posizione `pos` della Symbol Table (eventualmente aggiungendo altre informazioni).
3. Genera il token `<name, pos>`, dove `name` è il nome della categoria lessicale del lexeme.
4. Spedisce il token al parser.

Cos'altro può fare il Lexer? Oltre alla generazione dei token, il Lexer si occupa di:

- Rimuovere i commenti dal codice sorgente.
- Rimuovere i caratteri di spazio (blank, newline, tab, ecc.).
- Tenere il conto dei newline visti, in modo da fornire il numero di linea esatto per i messaggi di errore.
- Effettuare il preprocessing delle macro.

2.1.2 Categorie Lessicali e Symbol Table

Nel testo di riferimento, anche le Categorie Lessicali (CL) sono spesso chiamate token. Esempi tipici includono:

1. Ogni **parola chiave** del linguaggio ha un token dedicato (es. `if`, `while`).
2. Token per gli **operatori**, di solito divisi in classi (`arop`, `relop`, `assign`).
3. Un token unico per tutti gli **identificatori**, che includono variabili e nomi di funzione (`Id`).
4. Token dedicati per le **costanti numeriche** (`num`) e per le **stringhe** (`literal`).
5. Ogni **simbolo di punteggiatura** ha un token dedicato (di solito è il simbolo stesso).

Informazioni nella Symbol Table

Non tutti i token necessitano dello stesso livello di dettaglio all'interno della SymTab:

- **Nessuna informazione necessaria:** Parole chiave e simboli di punteggiatura.
- **Solo lexeme:** Operatori e costanti.
- **Informazioni multiple:** Gli identificatori necessitano del lexeme, del tipo e della linea di codice (per eventuali errori).

$\Rightarrow$ Di conseguenza, la SymTab deve poter contenere valori di grandezza variabile. La soluzione più semplice in linguaggio C è utilizzare locazioni di tipo `void *`.

2.1.3 Caratteri di Lookahead

Supponiamo di avere un lexeme ℓ che finisce in posizione i dell'input. Quanti caratteri dopo la posizione i sono necessari per riconoscere ℓ? Nei linguaggi moderni (e ben progettati), di solito ne basta uno. Tuttavia, in linguaggi storici come il **FORTRAN**, la situazione era ben diversa. Fino al FORTRAN 90, gli spazi all'interno di un identificatore erano permessi e ignorati. Consideriamo questi due statement legali:

```
DO 5 I = 1,25
DO 5 I = 1.25
```

- Il primo è un **ciclo DO**: `5` è la label dell'ultimo statement del corpo del ciclo, e `I = 1,25` indica che si itera su `I` da 1 a 25.
- Il secondo è un **assegnamento**: il valore da assegnare è 1.25 e la variabile destinataria si chiama `DO5I`.

In questo caso, il Lexer deve arrivare a leggere la virgola (`,`) o il punto (`.`) per poter riconoscere correttamente il lexeme. Per distinguere la parola chiave `DO` dalla variabile `DO5I`, deve essere letto un numero di caratteri $O(|labels| + |integers|)$ dopo il lexeme stesso. $\Rightarrow$ Questo dimostra che un buon design per un linguaggio di programmazione parte già dal lessico.

2.2 Stringhe, Linguaggi ed Espressioni Regolari

2.2.1 Terminologia per le Stringhe

- **Prefissi e Suffissi:** Parti iniziali o finali di una stringa s.
- **Sottostringa:** Si ottiene rimuovendo un prefisso e un suffisso da s. (Sono detti *propri* se non sono né la stringa vuota ϵ né uguali ad s stessa).
- **Sottosequenza:** Si ottiene rimuovendo caratteri da s in posizioni non necessariamente consecutive.

2.2.2 Linguaggi di Stringhe e Operatori

Un **alfabeto** Σ è un insieme finito di simboli. Un **linguaggio** su Σ è un insieme numerabile di stringhe di simboli in Σ. Anche l'insieme vuoto $\emptyset$ e l'insieme $\{\epsilon\}$ sono linguaggi.

Concatenazione

Date due stringhe x e y, xy denota la loro **concatenazione**. Vale l'identità: $s\epsilon = \epsilon s = s$. L'esponenziazione di stringhe è definita come $s^0 = \epsilon$ e $s^i = s^{i-1}s$.

Operatori su linguaggi

Dati due linguaggi L e M, definiamo i seguenti **operatori su linguaggi**:

- **Unione:** $L \cup M = \{s \mid s \in L \vee s \in M\}$
- **Concatenazione:** $LM = \{st \mid s \in L \wedge t \in M\}$
- **Chiusura di Kleene (star):** $L^* = \bigcup_{i=0}^{\infty} L^i$
- **Chiusura positiva:** $L^+ = \bigcup_{i=1}^{\infty} L^i$

Esempio (Identificatori): Se $L = \{A, \ldots, Z, a, \ldots, z, _\}$ e $D = \{0, \ldots, 9\}$, allora il linguaggio definito da $L(L \cup D)^*$ rappresenta perfettamente la categoria lessicale `Id` in C, C++, Java, ecc.

2.2.3 Espressioni Regolari (ER)

Espressioni Regolari

Gli operatori sui linguaggi sono adatti a definire le categorie lessicali, ma necessitano di una notazione compatta. Dato un alfabeto Σ e un'espressione regolare (ER) r, $L(r)$ denota il linguaggio definito da r.

La definizione è ricorsiva:

1. **Base:** ϵ è una ER e $L(\epsilon) = \{\epsilon\}$. Per ogni $a \in \Sigma$, a è una ER e $L(a) = \{a\}$.
2. **Induzione:** Siano r e s due ER:
   - $(r)|(s)$ è una ER e $L((r)|(s)) = L(r) \cup L(s)$
   - $(r)(s)$ è una ER e $L((r)(s)) = L(r)L(s)$
   - $(r)^*$ è una ER e $L((r)^*) = (L(r))^*$

- (r) è una ER e $L((r)) = L(r)$

Due ER r, s sono *equivalenti* $(r = s)$ se $L(r) = L(s)$. **Priorità degli operatori:** $| <$ concatenazione $< *$. Sia l'operatore $|$ che la concatenazione sono associativi a sinistra.
Esempi:

- ER per l'insieme $\{ab, aa, ba, bb\}$: $(a|b)(a|b)$
- ER per stringhe con numero generico di a e b: $(a|b)^*$ oppure $(a^*b^*)^*$
- ER per $\{a, b, ab, aab, aaab, \dots\}$: $a|a^*b$

Leggi Algebriche per le ER

- **Commutatività di $|$:** $r|s = s|r$
- **Associatività di $|$:** $r|(s|t) = (r|s)|t$
- **Associatività della concatenazione:** $r(st) = (rs)t$
- **Distributività:** $r(s|t) = rs|rt$ e $(s|t)r = sr|tr$
- **Identità per la concatenazione:** $\epsilon r = r\epsilon = r$
- **Idempotenza della chiusura:** $r^{**} = r^*$

2.2.4 Definizioni Regolari ed Estensioni

Definizione Regolare

Una **Definizione Regolare (DR)** è una lista di assegnamenti $d_1 \to r_1, \dots, d_n \to r_n$, dove tutti i d_i sono nomi distinti (non appartenenti a Σ) e r_i è un'espressione regolare costruita sull'alfabeto $\Sigma \cup \{d_1, \dots, d_{i-1}\}$.

Sono molto più leggibili per linguaggi complessi e vengono usate in strumenti come **Lex** per specificare il Lexer.

Esempio Identificatori:

```
letter_ → A | ... | Z | a | ... | z | _
digit   → 0 | ... | 9
id      → letter_ ( letter_ | digit )*
```

Esempio Numeri (Interi, Decimali ed Esponenziali):

```
digit       → 0 | ... | 9
digits      → digit digit*
optFraction → . digits | ϵ
optExponent → ( E ( + | - | ϵ ) digits )
              | ϵ
number      → digits optFraction optExponent
```

Estensioni per le ER Per rendere le definizioni ancora più succinte, sono stati introdotti operatori aggiuntivi (che però non aumentano il potere espressivo originale delle ER):

- **Positive closure (+):** Indica una o più occorrenze. $L(r^+) = (L(r))^+$. Ha la stessa precedenza di $*$.

- **Zero o una (?):** Operatore unario postfisso. Indica zero o un'occorrenza: $r? = (r|\epsilon)$. Stessa precedenza di $*$ e $+$.
- **Classi di caratteri ([ ]):** Sostituisce i blocchi in *or*. Ad esempio, `A | B | ... | Z` diventa il range `[A-Z]`.

2.3 Automi a Stati Finiti

Le ER sono ottime per *definire* i linguaggi, ma inutili per stabilire computazionalmente l'appartenenza di una stringa ad essi. Per costruire un "riconoscitore", si utilizzano gli Automi a Stati Finiti, che possono essere **Non-Deterministici (NFA)** o **Deterministici (DFA)**. Entrambi definiscono la stessa esatta classe: i **Linguaggi Regolari**.

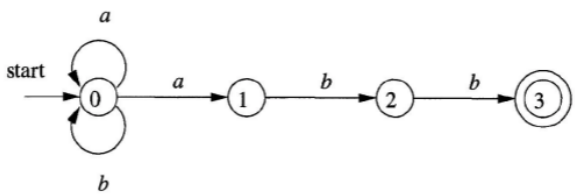
NFA: Automa Non-Deterministico

Un NFA è una quintupla $N = (\Sigma, S, \delta, s_0, F)$ dove:
- Σ: Alfabeto di input (finito), con $\epsilon \notin \Sigma$.
- S: Insieme finito degli stati.
- δ: Funzione di transizione, $\delta : S \times (\Sigma \cup \{\epsilon\}) \to \mathcal{P}(S)$, dove $\mathcal{P}(S)$ è l'insieme delle parti di S. In alternativa, può essere vista come una relazione $\delta \subseteq (S \times (\Sigma \cup \{\epsilon\})) \times S$.
- $s_0 \in S$: Stato di partenza.
- $F \subseteq S$: Insieme degli stati finali o di accettazione.

Fonti del non-determinismo: la possibilità di transizioni su ϵ e la possibilità di avere più archi uscenti dallo stesso stato con la stessa etichetta.

2.3.1 Rappresentazione e Accettazione NFA

Un NFA può essere rappresentato tramite una **Transition Table** o graficamente tramite un **Transition Graph** $G = (V, E)$, un grafo orientato in cui i vertici sono gli stati e gli archi sono etichettati con i simboli di $\Sigma \cup \{\epsilon\}$. Gli stati di accettazione hanno un doppio cerchio.



STATE	a	b	ϵ
0	$\{0,1\}$	$\{0\}$	$\emptyset$
1	$\emptyset$	$\{2\}$	$\emptyset$
2	$\emptyset$	$\{3\}$	$\emptyset$
3	$\emptyset$	$\emptyset$	$\emptyset$

Figura 2: Automa Non-Deterministico (NFA)

Un NFA **accetta una stringa** x se esiste un cammino orientato $p = u_0, u_1, \ldots, u_k$ tale che:

1. $u_0 = s_0$ (parte dallo stato iniziale).
2. $u_k \in F$ (termina in uno stato finale).
3. Sia l_i l'etichetta dell'arco in p, e sia ℓ la sottosequenza ottenuta rimuovendo tutti gli ϵ: la sequenza ℓ deve essere esattamente uguale a x.

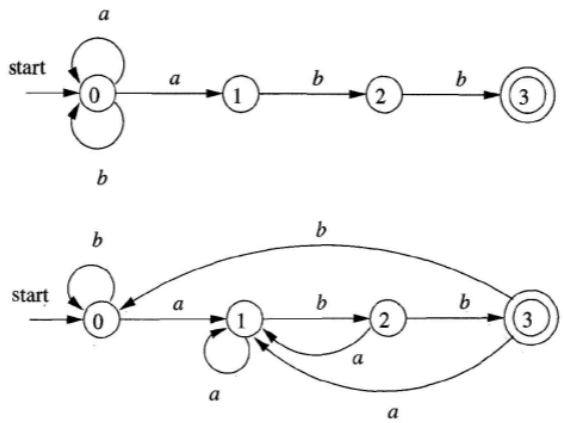
DFA: Automa Deterministico

Un DFA è una quintupla $D = (\Sigma, S, \delta, s_0, F)$. La grande differenza rispetto all'NFA risiede unicamente nella definizione della funzione δ:

$$\delta : S \times \Sigma \to S$$

- Non esistono transizioni vuote (ϵ).
- δ mappa verso un singolo stato S, non verso un insieme di stati $\mathcal{P}(S)$.

2.3.2 Differenze Fondamentali tra DFA e NFA



Le regole di **accettazione** per un DFA sono quasi identiche a quelle dell'NFA, con la differenza che non essendoci transizioni ϵ, la sequenza esatta delle etichette degli archi deve comporre la stringa x. Data la sua natura deterministica, il cammino p per una stringa x in un DFA **esiste sempre ed è unico**. Se al termine del cammino ci si trova in F, x è accettata; altrimenti no. La **differenza operativa centrale** emerge nella definizione della funzione δ, che è totale per entrambi:

- **Nel DFA:** Per ogni stato $s \in S$ e carattere $a \in \Sigma$, esiste *sempre* esattamente un arco in uscita. La simulazione è lineare: basta implementare una procedura `move(s, c)` accoppiata a `nextChar()`.
- **Nell'NFA:** Il codominio è l'insieme delle parti $\mathcal{P}(S)$. Questo significa che $\delta(s, h)$ può restituire l'insieme vuoto $\emptyset$. Se l'automa si trova in uno stato s per cui non ci sono transizioni valide per il carattere letto h, l'NFA si "pianta" e la Transition Table conterrà dei valori nulli. Simularlo richiede di tracciare percorsi multipli in parallelo.

2.4 Convertire un NFA in un DFA

Come vedremo, data un'Espressione Regolare (ER) e, costruire l'NFA N_e tale per cui $L(e) = L(N_e)$ è abbastanza semplice. Se sapessimo convertire N_e in un DFA D_e tale per cui $L(N_e) = L(D_e)$ avremmo un riconoscitore deterministico per l'ER e. L'algoritmo **Subset Construction** fa esattamente questo.

2.4.1 Alcune definizioni

Dato un NFA $N = (\Sigma, S, \delta, s_0, F)$ definiamo tre insiemi di stati:

1. **ϵ-closure(s)**: per $s \in S$, contiene s e gli stati di N raggiungibili da s con cammini i cui archi sono etichettati solo con ϵ.
2. **ϵ-closure(T)**: per $T \subseteq S$, è definito come

$$\bigcup_{s \in T} \epsilon\text{-closure}(s)$$

   Per calcolarlo, si può usare una visita BFS o DFS sul grafo delle transizioni, inizializzando la coda dei nodi a T e considerando solo gli archi con etichetta ϵ.
3. **move(T, a)**: dati $T \subseteq S$ e $a \in \Sigma$, è definito come $\{s \in S \mid \exists g \in T$ tale per cui $s \in \delta(g, a)\}$.

2.4.2 Subset Construction: idea di base

Consideriamo un NFA $N = (\Sigma, S, \delta, s_0, F)$ e sia D il DFA da costruire. Per $a \in \Sigma$, qual è l'insieme di tutti gli stati in cui N si può trovare dopo aver letto a a partire da s_0? La risposta è ϵ-closure(move(ϵ-closure$(s_0), a)$).

La procedura segue questi passi:

1. ϵ-closure(s_0) è lo stato iniziale di D, chiamiamolo S_0.
2. ϵ-closure(move(S_0, a)) è lo stato di D a cui si arriva a partire da S_0 leggendo a nell'input.
3. Se ϵ-closure(move$(S_0, a)) \neq S_0$, chiamiamolo S_1.
4. Per $b \in \Sigma$, qual è l'insieme degli stati in cui N si può trovare a partire dagli stati in S_0? ϵ-closure(move(S_0, b)).
5. Se ϵ-closure(move$(S_0, b)) \neq S_0$ e $\neq S_1$ allora è un altro stato di D, chiamiamolo S_2.

Si prosegue così:

1. Si sceglie uno stato S_i di D non ancora esaminato.
2. $\forall x \in \Sigma$:
   (a) Si calcola ϵ-closure(move(S_i, x)).
   (b) Se il nuovo insieme è diverso dai precedenti allora è un nuovo stato di D.

2.4.3 Il DFA finale

Il DFA $D = (\Sigma, S_D, \delta_D, S_0, F_D)$ ottenuto da $N = (\Sigma, S, \delta, s_0, F)$ tramite Subset Construction è così definito:

- S_D è definito dai Dstates.
- δ_D è definita dalle Dtran.
- S_0 è ϵ-closure(s_0).
- F_D contiene ogni $s \in S_D$ che contenga almeno un $f \in F$.

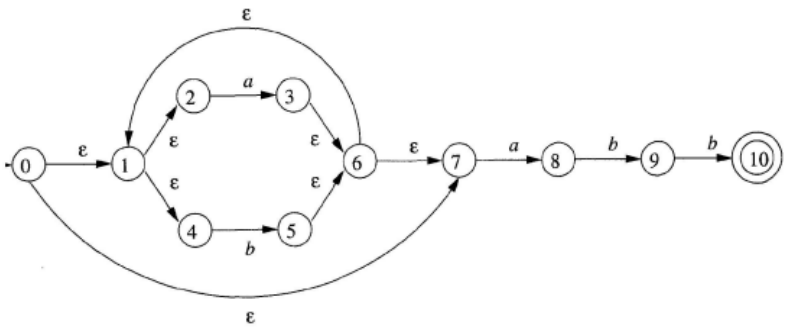
2.4.4 Esempio di Subset Construction

Costruiamo l'automa per $(a|b)^*abb$.

1. Stato iniziale di D: $A = \epsilon\text{-closure}(0) = \{0, 1, 2, 4, 7\}$
2. $\epsilon\text{-closure}(\text{move}(A, a)) = \epsilon\text{-closure}(\{3, 8\}) = \{1, 2, 3, 4, 6, 7, 8\} = B$
3. $\epsilon\text{-closure}(\text{move}(A, b)) = \epsilon\text{-closure}(\{5\}) = \{1, 2, 4, 5, 6, 7\} = C$
4. $\epsilon\text{-closure}(\text{move}(B, a)) = \epsilon\text{-closure}(\{3, 8\}) = B$
5. $\epsilon\text{-closure}(\text{move}(B, b)) = \epsilon\text{-closure}(\{5, 9\}) = \{1, 2, 4, 5, 6, 7, 9\} = D$
6. $\epsilon\text{-closure}(\text{move}(C, a)) = \epsilon\text{-closure}(\{3, 8\}) = B$
7. $\epsilon\text{-closure}(\text{move}(C, b)) = \epsilon\text{-closure}(\{5\}) = C$
8. $\epsilon\text{-closure}(\text{move}(D, a)) = \epsilon\text{-closure}(\{3, 8\}) = B$
9. $\epsilon\text{-closure}(\text{move}(D, b)) = \epsilon\text{-closure}(\{5, 10\}) = \{1, 2, 4, 5, 6, 7, 10\} = E$
10. $\epsilon\text{-closure}(\text{move}(E, a)) = \epsilon\text{-closure}(\{3, 8\}) = B$
11. $\epsilon\text{-closure}(\text{move}(E, b)) = \epsilon\text{-closure}(\{5\}) = C$

Nel DFA finale:

- A è lo stato iniziale.
- E è l'unico stato di accettazione (è l'unico che contiene 10).
- Nota: A e C hanno le stesse transizioni e quindi possono essere fusi.



2.4.5 Complessità del Subset Construction

Gli stati di D sono sottoinsiemi di S di N. Di conseguenza D può avere al più 2^n stati (con $n = |S|$). Esistono casi in cui D ha effettivamente $\Omega(2^n)$ stati? Sì, si può dimostrare che al caso pessimo il numero di stati prodotti è $\Theta(2^n)$. **Esempio worst case:** consideriamo il linguaggio su $\{a, b\}$, parametrico su n, che contiene stringhe di almeno n caratteri, e il cui n-esimo simbolo a partire dall'ultimo è a. Ovvero l'ER: $L_n = (a|b)^* a(a|b)^{n-1}$. Questo NFA ha $n + 1$ stati, ma qualsiasi DFA D per L_n ha almeno 2^n stati. Se D avesse $< 2^n$ stati, per il principio della piccionaia, esisterebbero due stringhe x, y distinte, lunghe n, che finirebbero nello stesso stato s. Considerando l'ultimo simbolo in cui x e y differiscono (sia esso il d-esimo a partire da destra) e concatenando una stringa z lunga $n - d$, avremmo che xz e yz finiscono nello stesso stato di D, pur dovendo avere esiti di accettazione opposti (contraddizione). **Analisi del costo:**

- Il corpo del ciclo `for` viene eseguito $O(V(D) + E(D))$ volte, dove $V(D) = |S_D|$ e $E(D) = |\Sigma||S_D|$.
- Il costo di ogni esecuzione del corpo del `for` è $O(V(N) + E(N))$.
- Quindi la complessità totale del Subset Construction è $O((V(N) + E(N))|\Sigma||S_D|)$.

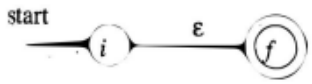
2.5 Simulare un NFA

Data una stringa in input x e un NFA N, sia D il DFA che il Subset Construction produrrebbe da N. La simulazione calcola gli stati di D come nel Subset Construction, ma produce solo quelli su cui D passerebbe se lo usassimo su x. Poiché non li memorizza, può ricalcolare stati già calcolati. In totale calcolerà (o ricalcolerà) sempre $|x| + 1$ stati di D. La complessità complessiva della simulazione dell'NFA è $O(|x|(V(N) + E(N)))$.

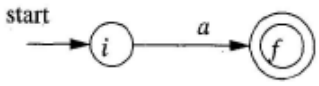
2.6 Da ER a NFA: Algoritmo di McNaughton, Yamada e Thompson

Data un'ER e, l'algoritmo produce l'NFA N_e tale per cui $L(e) = L(N_e)$. È definito ricorsivamente, basandosi sull'Abstract Tree dell'ER:

- **Caso base per** ϵ: due nuovi stati i e f connessi da una transizione ϵ.

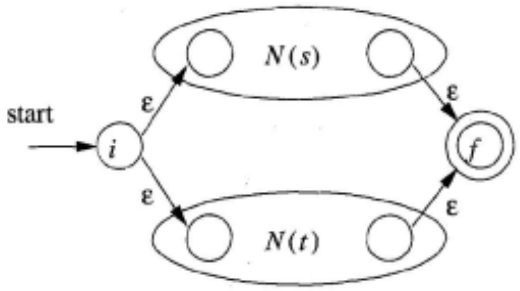


- **Caso base per ogni** $a \in \Sigma$: due nuovi stati i e f connessi da una transizione a.

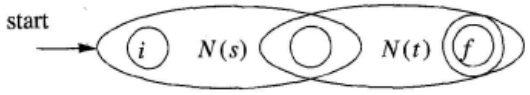


- **Caso induttivo per l'unione (|)**: Siano $N(s)$ e $N(t)$ gli NFA delle sottoespressioni. Si creano due nuovi stati i e f. Da i partono due ϵ-transizioni verso gli stati

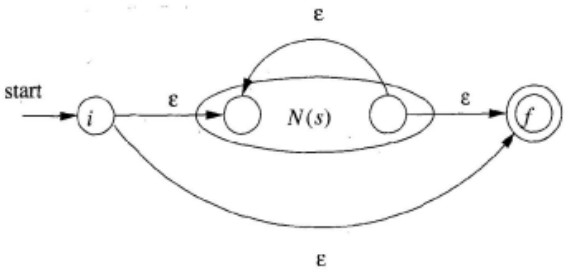
iniziali di $N(s)$ e $N(t)$. Dagli stati di accettazione di $N(s)$ e $N(t)$ partono due ϵ-transizioni verso f.



- **Caso induttivo per la concatenazione**: Lo stato di accettazione (unico) di $N(s)$ e quello iniziale di $N(t)$ vengono fusi insieme. i diventa lo stato iniziale di $N(s)$ e f lo stato di accettazione di $N(t)$.



- **Caso induttivo per la chiusura di Kleene ($*$)**: Si creano due nuovi stati i e f. Da i partono ϵ-transizioni verso lo stato iniziale di $N(s)$ e verso f. Dallo stato finale di $N(s)$ partono ϵ-transizioni verso lo stato iniziale di $N(s)$ e verso f.



- **Caso induttivo per le parentesi**: Si usa direttamente $N(s)$ senza cambiamenti.

Note sull'automa generato:

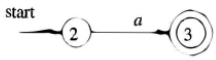
- Ha un numero di stati che è al più il doppio del numero totale di operatori e operandi in e.
- Ha un solo stato di accettazione senza transizioni uscenti.
- Lo stato iniziale non ha transizioni entranti.
- Ogni stato ha o una transizione uscente su un simbolo in Σ, o al più due transizioni uscenti entrambe su ϵ.

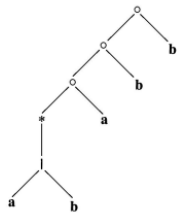
E l'Abstract Tree? L'algoritmo assume di avere a disposizione l'albero sintattico. Strumenti come Lex necessitano di un lexer interno e un parser per costruire l'abstract tree dell'ER fornita in input.

2.6.1 Esempio

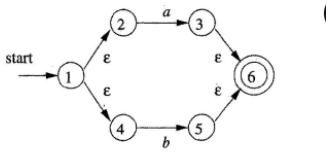
Usiamo l'algoritmo di McNaughton, Yamada e Thompson per convertire l'espressione regolare $(a|b)^*abb$. Assumiamo di avere a disposizione l'abstract tree. Il cerchio rappresenta l'operatore di concatenazione.

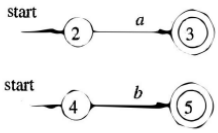
1. Visitando in profondità arriviamo alla foglia più a sinistra. $a \in \Sigma$ quindi applichiamo il secondo caso base:



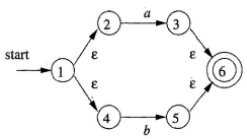


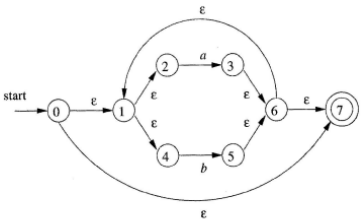
2. Risaliamo verso |, e scendiamo nel figlio destro. $b \in \Sigma$ quindi applichiamo il secondo caso base:



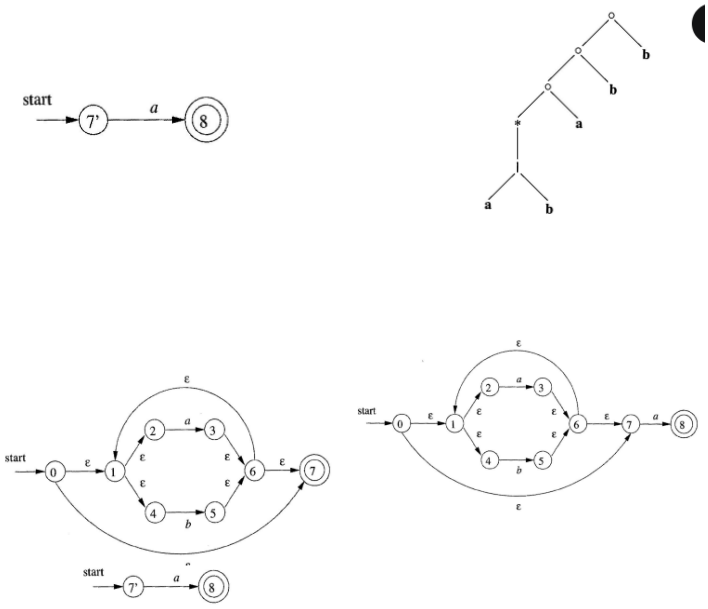


3. Risaliamo verso |. Combiniamo i due automi costruiti finora usando il primo caso ricorsivo:

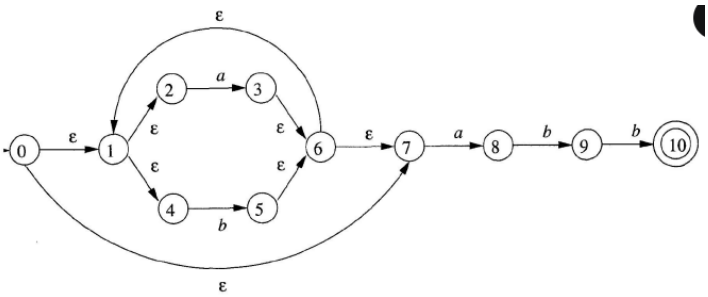




4. Risaliamo verso $*$. Sull'automa costruito nel passo precedente applichiamo il terzo caso ricorsivo:



5. Risaliamo verso $\circ$ e scendiamo nel figlio destro. È una foglia e $a \in \Sigma$ quindi applichiamo il secondo caso base:
6. Risaliamo verso $\circ$ e scendiamo nel figlio destro. Combiniamo i due automi costruiti finora usando il secondo caso ricorsivo:



7. Andando avanti in questa maniera arriviamo a quell'automa che avevamo visto in precedenza.

2.7 Programmi che usano ER: Confronto Costi

Esistono due approcci principali per processare un'ER r su una stringa x:

1. ER $\Rightarrow$ NFA $\Rightarrow$ DFA $\Rightarrow$ Simulazione DFA.
2. ER $\Rightarrow$ NFA $\Rightarrow$ Simulazione NFA.

Costruire un intero DFA non è sempre conveniente. In un compilatore conviene (il DFA è costruito una volta e usato sempre), mentre per tool come `grep` o text editor (ricerca singola) potrebbe convenire simulare un NFA.

Approccio	Costo iniziale	Costo per stringa
2 (NFA)	$O(\|r\|)$	$O(\|r\|\|x\|)$
1 (DFA - tipico)	$O(\|r\|^3)$	$O(\|x\|)$
1 (DFA - pessimo)	$O(2^{\|r\|})$	$O(\|x\|)$

2.8 Lexical Analysis: dagli Automi al Lexer

2.8.1 Struttura di un lexer (generato)

Un lexer generato automaticamente è composto da due parti principali:

- **Simulatore di automi a stati finiti**: È una parte fissa di codice (cioè non dipende dalle specifiche date al generatore). Riceve l'automa da simulare sotto forma di tabella.
- **La parte prodotta dal generatore**: Contiene la tabella delle transizioni e le azioni da eseguire quando si verifica un match.

2.8.2 Riconoscimento vs Lexing

Il problema risolto da un automa è basilare:

- È definito su una singola Espressione Regolare (ER).
- Prende in input una stringa (già nota).
- Restituisce "sì" o "no".

Il problema risolto dal Lexer è molto più complesso:

- È definito su un *insieme* di ER I.
- Prende in input un intero file F.
- Deve spezzare F in singole stringhe, ognuna delle quali è un match per qualche ER in I.
- Non è detto che una stringa sia un match per una sola ER.
- Stringhe che iniziano nella stessa posizione in F ma di lunghezze differenti possono ciascuna essere un match per una particolare ER in I.
- Per ogni singola stringa riconosciuta deve eseguire qualche tipo di azione.

2.8.3 Specifiche per un generatore: ER multiple

Tipicamente le specifiche date ad un generatore (come Lex) saranno di questo tipo:

```
p1 {frag1}
p2 {frag2}
...
pn {fragn}
```

Si tratta di una lista di ER $p_1, \ldots, p_n$, ognuna rappresentante una categoria lessicale (identificatori, costanti numeriche, ecc.). Ogni ER ha associata un'azione composta da

un frammento di codice da eseguire in caso di match. Il primo passo che il generatore compie è quello di costruire il corrispondente NFA $N(p_i)$ per ogni ER p_i. Dopodiché unisce tutti gli NFA in un singolo automa:

- Un nuovo stato iniziale globale è connesso con ϵ-transizioni allo stato iniziale di ogni $N(p_i)$.
- Al contrario della costruzione standard di McNaughton, Yamada e Thompson, questo NFA globale ha n stati di accettazione, uno per ogni ER.

2.8.4 Conflitti inter-ER e intra-ER

Consideriamo questa specifica (le azioni non sono importanti per ora):

```
a       {frag1}
abb     {frag2}
a*b+    {frag3}
```

Ci sono due problemi principali con questa specifica:

- **Conflitto inter-ER**: Esistono stringhe che sono un match sia per la seconda che per la terza ER (es. `abb`). Conflitti di questo genere sono risolti con regole di priorità. Tipicamente (ad esempio in Lex) la regola è: in caso di matching multipli si sceglie l'ER più in alto nella lista.
- **Conflitto intra-ER**: Esistono stringhe che hanno prefissi multipli che sono un match per la terza ER. Ad esempio, per la stringa `aabbb`, i prefissi `aab`, `aabb` e `aabbb` sono tutti match validi per `a*b+`. Conflitti di questo tipo sono tipicamente risolti scegliendo il prefisso più lungo fra tutti quelli in match (*longest matching prefix*).

2.8.5 Lexer basato su NFA

Assumiamo che la costruzione del Lexer si fermi all'NFA. Il Lexer sarà basato sull'algoritmo di simulazione di un NFA:

1. La simulazione dell'NFA inizia a leggere dalla posizione corrente.
2. L'insieme di stati S viene calcolato per il carattere corrente.
3. L'insieme S viene salvato prima di leggere il carattere successivo.
4. ...

X. Ad un certo punto la simulazione leggerà un carattere per il quale $S = \emptyset$ (il cosiddetto "momento X"):
   (a) Si scandiscono gli insiemi di stati salvati $S_t, S_{t-1}, \ldots, S_1$ in ordine inverso.
   (b) Si trova il primo S_i contenente almeno uno stato di accettazione.
   (c) Fra tutti gli stati di accettazione in S_i si sceglie quello corrispondente all'ER p_j posizionata più in alto nelle specifiche.
   (d) Si esegue l'azione associata a p_j.

(e) Si riparte dal punto 1 per riconoscere il lexeme successivo.

Esempio di esecuzione su stringa `aaba`: Prendiamo la specifica di prima (`a`, `abb`, `a*b+`) e simuliamo l'NFA sulla stringa `aaba`.

- Durante la lettura, l'insieme degli stati viene calcolato e salvato per ogni prefisso (`a`, `aa`, `aab`).
- Quando si legge l'ultima `a` (diventando `aaba`), la simulazione si blocca ($S = \emptyset$) perché nessuna ER accetta una `a` dopo una `b` in quel contesto.
- Scansionando a ritroso, il lexeme `aab` viene scelto.
- `aab` appartiene all'ER `a*b+` e quindi il codice in `{frag3}` viene eseguito.
- Sebbene anche l'ER `a` abbia un match con il primissimo carattere `a` (ed è più in alto nelle specifiche), lo stato di accettazione corrispondente si trova in un S_i precedente rispetto a quello di `aab`. La regola del *longest matching prefix* favorisce quindi `a*b+`.

E il "momento X"? Siamo sicuri che la simulazione arrivi sempre al "momento X" (cioè $S = \emptyset$)? Dipende da come abbiamo definito le specifiche. L'alfabeto per l'intero lessico del linguaggio è $\Sigma = \Sigma_1 \cup \cdots \cup \Sigma_n$, dove Σ_i è l'alfabeto della singola ER p_i. Poiché gli alfabeti tipicamente non sono uguali, questo garantisce che l'NFA si "pianti" prima o poi. Ad esempio, se abbiamo categorie per costanti numeriche, identificatori, operatori aritmetici e spazi:

- L'NFA per i numeri si "pianta" quando legge lettere o operatori.
- L'NFA per gli identificatori si "pianta" quando legge operatori o spazi.

2.8.6 Lexer con NFA $\Rightarrow$ DFA

Assumiamo ora che il generatore del Lexer converta l'NFA N in un DFA D. Come gestiamo le due politiche per i conflitti?

- **Conflitti Inter-ER**: Assumiamo che un prefisso dell'input sia un match per k ER delle specifiche. I corrispondenti stati di accettazione degli automi originali sono contenuti dentro lo stesso macro-stato s del DFA D. Se la simulazione finisce su s, troviamo l'ER con stato di accettazione contenuto in s che sia posizionata più in alto nelle specifiche.
- **Conflitti Intra-ER**: Dobbiamo rispettare il *longest matching prefix*. I DFA "regolari" non sono adatti perché, se fermiamo D appena arriva su uno stato di accettazione, la computazione si fermerà al suffisso più corto (MALE!). Se non facciamo fermare D, la computazione si leggerà tutto il testo senza mai fermarsi (PEGGIO!).

Soluzione per il Lexer con DFA: Per usare nel Lexer un DFA D costruito tramite Subset Construction, dobbiamo:

1. Lasciar continuare la computazione anche quando si arriva a stati in F_D (per la regola del *longest matching prefix*).
2. Salvare tutti gli stati di D da cui si passa.

3. Ad un certo momento, si ripercorrono gli stati salvati all'inverso finché troviamo un macro-stato $s \in S_D$ tale che esista almeno uno stato originario $t \in F_N$ con $t \in s$.

4. Se s contiene più di un $t \in F_N$, scegliamo quello corrispondente all'ER più in alto.

Il momento esatto in cui fermarsi e tornare indietro dipende dai **Dead State** (uno stato $s \notin F$ tale che $\forall a \in \Sigma$, $\delta(s, a) = s$). Nel Subset Construction, un dead state nasce se l'insieme ϵ-closure(move(S, a)) risulta vuoto.

- Se **manteniamo** i dead state in D, il momento in cui fermarsi è appena si entra nel dead state (la lettura dell'input viene sospesa).
- Se **eliminiamo** i dead state da D, il momento in cui fermarsi è quando l'automa si "pianta" su uno stato (ovvero non esiste una transizione per il simbolo corrente).

2.8.7 L'operatore "Lookahead"

In alcuni linguaggi può essere necessario leggere più di un carattere dopo la fine di un lexeme per poterlo riconoscere. Ad esempio, in FORTRAN le parole chiave come `IF` o `THEN` non sono riservate. Si possono avere due casi:

```
IF(i,j)=34        // Assegnamento ad un array
IF(cond) THEN ... // Costrutto condizionale
```

Il Lexer, per capire se si tratta di un costrutto o di un array, deve continuare a leggere finché non trova `=` (per l'array) o `T` (per l'IF). Per ovviare a questo problema esiste l'operatore di **lookahead**, denotato in Lex da `/`. In un'ER nella forma `r1 / r2`:

- `r1` è l'ER che definisce la categoria lessicale vera e propria.
- `r2` è l'ER che descrive il pattern di contesto successivo.

Per il FORTRAN l'espressione in sintassi Lex sarebbe `IF / \( .* \) T`. **Implementazione dell'operatore Lookahead:** La conversione in NFA è semplice: `r1` e `r2` vengono tradotte normalmente, e `/` viene trattato come una ϵ-transizione. La simulazione però cambia. Se l'NFA riconosce la stringa xy come match di `r1 r2` (x per `r1` e y per `r2`), la vera "fine del lexeme" non si trova nello stato finale s_t, ma in uno stato intermedio s (quello prima della ϵ-transizione generata dal `/`). Durante la costruzione, questo stato s viene "marcato". La simulazione, una volta raggiunto s_t (avendo letto tutto xy), ripercorrerà all'indietro gli insiemi di stati salvati fino a trovare quello più recente che contenga lo stato marcato s.

2.8.8 Minimizzazione degli stati di un DFA

Abbiamo visto che per una stessa espressione regolare (es. `(a|b)*abb`) possono esistere più DFA validi, ma con numero di stati diverso. Intuitivamente, stati che hanno esattamente le stesse transizioni verso gli stessi stati possono essere fusi. **Definizioni:** Dati un DFA $D = (\Sigma, S, \delta, s_0, F)$, una stringa x e due stati s, t, definiamo le macchine parziali D_s (con stato iniziale s) e D_t (con stato iniziale t). Diremo che x **distingue** s da t se l'esito della computazione di x differisce (es. x è accettata partendo da s ma non da t, o viceversa). Diremo che s è **distinguibile** da t se esiste almeno una stringa che li distingua. Altrimenti, sono **indistinguibili** (ed equivalenti).

Idee di base dell'algoritmo di minimizzazione: L'algoritmo trova dei macro-insiemi di stati $S_1, S_2, \ldots, S_k$ che siano mutuamente disgiunti, coprano tutti gli stati originali, e in cui ogni coppia interna sia indistinguibile, mentre ogni coppia tra insiemi diversi sia distinguibile. Si procede per *approssimazioni successive* partendo da una partizione Π:

1. ϵ distingue sempre ogni stato $s \in F$ da ogni stato $t \in S - F$. Quindi si parte con $\Pi = \{F, S - F\}$.
2. Se in un gruppo di Π ci sono stati che, leggendo un certo simbolo c, finiscono in due gruppi distinti di Π, quel gruppo viene "spaccato" in due.
3. Si ripete il raffinamento finché la partizione si stabilizza (non ci sono più discrepanze).
4. Per ogni insieme finale in Π_{final}, si sceglie un singolo "rappresentante". Questi rappresentanti diventeranno i nodi del nuovo automa minimizzato D_{min}.

Esempio di minimizzazione: Prendiamo l'automa originale visto per `(a|b)*abb` con stati A, B, C, D (non finali) ed E (finale).

1. Partizione iniziale: $\Pi = \{\{A, B, C, D\}, \{E\}\}$.
2. Guardando $\{A, B, C, D\}$ col simbolo b, gli stati A, B e C rimangono in $\{A, B, C, D\}$, ma lo stato D transita verso E (che è nell'altro gruppo). Quindi dividiamo: $\Pi = \{\{A, B, C\}, \{D\}, \{E\}\}$.
3. Guardando $\{A, B, C\}$ col simbolo b, A e C rimangono in $\{A, B, C\}$, ma B transita in D (che è in un gruppo diverso). Dividiamo ancora: $\Pi = \{\{A, C\}, \{B\}, \{D\}, \{E\}\}$.
4. Valutando $\{A, C\}$, per entrambi i simboli (a e b) finiscono nello stesso gruppo. La partizione è finale. A e C vengono fusi.

Nota sul dead state: l'algoritmo di minimizzazione può generare o mantenere un dead state. Se provassimo a rimuovere il dead state da D_{min} per avere meno stati, l'automa risultante riconoscerebbe lo stesso linguaggio ma *non* sarebbe più un DFA formale.

2.9 Generazione Automatica di Lexer: Lex

Lex è uno strumento standard per la generazione di lexer.

- I file sorgenti hanno l'estensione `.l`.
- Il risultato è un file C (`lex.yy.c`) contenente la funzione `yylex()`.
- `yylex()` è invocata tipicamente da un parser (es. generato da Yacc). Se non ci sono azioni con `return`, la funzione processa tutto l'input.
- Quando si riconosce un token, `yylex()` restituisce un intero che rappresenta la categoria lessicale, e carica eventuali attributi semantici nella variabile globale `yylval`.

Struttura di un file `.l`: È diviso in tre sezioni delimitate da `%%`:

```
dichiarazioni
%%
```

```
regole
%%
funzioni ausiliarie
```

1. **Dichiarazioni**: Include dichiarazioni in C racchiuse tra `%{` e `%}` (che vengono ricopiate testualmente in `lex.yy.c`), definizioni dei nomi dei Token (costanti numeriche) e definizioni regolari (es. `<nome> <expr reg>`).

2. **Regole**: Consistono in una serie di coppie `pattern {azione}`. I pattern usano le definizioni della sezione precedente e specificano il codice C da eseguire in caso di match (tipicamente termina con un `return`).

3. **Funzioni ausiliarie**: Funzioni C definite dall'utente utili per le azioni. Il lexer imposta automaticamente due variabili utilissime: `yytext` (puntatore all'inizio della stringa testuale del lexeme corrente) e `yyleng` (la lunghezza del lexeme corrente).

3

Analisi Sintattica e Parsing

3.1 Sintassi e Grammatiche Context-Free

Le Componenti di una Grammatica Context-Free

Una grammatica context-free è formalmente definita da quattro componenti principali:

1. **Simboli Terminali (T):** Costituiscono l'alfabeto di base, ovvero i simboli elementari che non possono essere ulteriormente sostituiti.
2. **Simboli Non-Terminali (N):** Fungono da "variabili sintattiche" e possono essere rimpiazzati applicando le regole di produzione.
3. **Produzioni (P):** Sono le regole di riscrittura espresse nella forma $\alpha \to \beta$, dove α e β sono stringhe sull'alfabeto $T \cup N$. Nella fattispecie delle grammatiche *context-free*, α è sempre rigorosamente un singolo simbolo non-terminale.
4. **Simbolo di Partenza:** Un simbolo specifico appartenente all'insieme N da cui ha inizio la derivazione.

Esempi di produzioni (con la stringa vuota ϵ):

```
list   → list + digit | list - digit | digit
digit  → 0 | 1 | ... | 9

call       → id ( optparams )
optparams  → params | ϵ
params     → params , param | param
param      → id
```

3.1.1 Convenzioni Notazionali

Poiché una grammatica context-free (CFG) viene solitamente presentata come una semplice lista di produzioni, è utile adottare delle convenzioni standard:

- **Per i terminali:** Si usano le prime lettere minuscole dell'alfabeto (a, b, c), operatori $(+, *)$, simboli di punteggiatura, singole cifre $(0, 1, 2)$ e stringhe in grassetto (es. **if**, **then**).
- **Per i non-terminali:** Si usano le prime lettere maiuscole (A, B, C). Il simbolo S, se presente, indica il simbolo iniziale. Spesso si usano anche stringhe in corsivo (es. *expr*, *stmt*).
- **Regole generali:** Le lettere finali maiuscole (X, Y, Z) indicano simboli generici in $N \cup T$. Le lettere finali minuscole (u, v, z) rappresentano stringhe di soli

terminali. Le lettere greche (α, β, γ) indicano stringhe miste in $N \cup T$. In assenza di indicazioni, il simbolo di partenza coincide con il non-terminale situato nella testa della prima produzione elencata.

3.2 Gerarchia e Tipi di Grammatiche

A seconda delle restrizioni applicate alla forma delle produzioni $\alpha \to \beta$, le grammatiche si dividono in diverse classi (secondo la Gerarchia di Chomsky).

Classificazione delle Grammatiche

Grammatiche Context-free (CFG)
Una grammatica $G = (N, T, P, S)$ è context-free se ogni produzione in P ha la forma:

$$A \to \beta$$

dove il lato sinistro è composto da un singolo non-terminale ($A \in N$) e il lato destro β è una stringa qualsiasi in $(N \cup T)^*$. "Context-free" significa che il non-terminale A può essere sostituito con β indipendentemente dal contesto in cui A si trova.
Grammatiche Regolari
Una grammatica è regolare (e genera linguaggi regolari, equiparabili alle Espressioni Regolari) se ogni produzione ha una di queste tre forme:

- $A \to aB$ (un terminale seguito da un non-terminale)
- $A \to a$ (un singolo terminale)
- $A \to \epsilon$ (la stringa vuota)

dove $A, B \in N$ e $a \in T$.
Grammatiche Context-sensitive
Una grammatica è context-sensitive se ogni produzione ha la forma:

$$\alpha A \beta \to \alpha \gamma \beta$$

dove $A \in N$, $\alpha, \beta \in (N \cup T)^*$ e $\gamma \in (N \cup T)^+$. Qui la sostituzione di A con γ può avvenire solo se A è circondato dallo specifico contesto α e β.

3.3 Derivazioni e Linguaggio

Il processo per generare una stringa a partire dal simbolo iniziale S si chiama derivazione. Il processo di derivazione consiste nel partire da una stringa α, individuare al suo interno un non-terminale A (quindi $\alpha = \beta A \gamma$), scegliere una produzione $A \to \delta$ e sostituire A con δ, ottenendo $\beta \delta \gamma$. Questo processo si reitera sui nuovi non-terminali finché non si raggiunge la forma desiderata.

Definizione Formale di Derivazione

Data una grammatica CF $G = (N, T, P, S)$, una derivazione è una sequenza di stringhe $\alpha_0, \alpha_1, \ldots, \alpha_k$ tale che per ogni passo i $(0 \leq i < k)$:

1. $\alpha_i = \gamma_i A_i \delta_i$
2. Esiste una produzione $A_i \to \theta_i \in P$
3. $\alpha_{i+1} = \gamma_i \theta_i \delta_i$

In questo caso, si afferma che α_k è derivabile da α_0, sinteticamente indicato con $\alpha_0 \Rightarrow^* \alpha_k$.

Definizione di Derivazione e Linguaggio

Data una grammatica generica $G = (N, T, P, S)$, una derivazione è una sequenza di stringhe $\alpha_0, \alpha_1, \ldots, \alpha_k$ tale che per ogni passaggio $0 \leq i < k$:

1. $\alpha_i = \gamma_i \beta_i \delta_i$ (ovvero individuiamo una sottostringa β_i)
2. Esiste una produzione $\beta_i \to \theta_i \in P$
3. Sostituiamo β_i per ottenere $\alpha_{i+1} = \gamma_i \theta_i \delta_i$

Notazioni:

- $\alpha \Rightarrow \alpha_k$: derivabile in esattamente 1 passo.
- $\alpha \xrightarrow{*} \alpha_k$: derivabile in 0 o più passi.
- $\alpha \xrightarrow{+} \alpha_k$: derivabile in 1 o più passi.

Se $S \xrightarrow{*} \alpha$, allora α è chiamata **sentential form** (forma sentenziale). Se una forma sentenziale è composta solo da terminali ($S \xrightarrow{*} u$ con $u \in T^*$), prende il nome di **sentenza**. Il **Linguaggio** $L(G)$ è l'insieme di tutte le sentenze generabili da G:

$$L(G) = \{u \mid u \in T^* \text{ e } S \xrightarrow{*} u\}$$

Linguaggio di una Grammatica Context-Free

Il **Linguaggio di una grammatica** $L(G)$ è l'insieme di tutte le stringhe u composte esclusivamente da simboli terminali, tali per cui u sia derivabile dal simbolo di partenza S (ossia $S \Rightarrow^* u$).

Esempi pratici di derivazione:

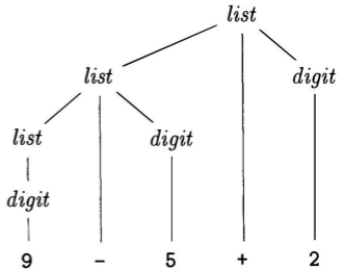
- La stringa `9 - 5 + 2` **appartiene** al linguaggio della grammatica `list`? Sì, attraverso i passaggi: `list` $\to$ `list + digit` $\to$ `list digit + digit` $\cdots \to$ `9 - 5 + 2`.
- La stringa `id ( id , )` **appartiene** al linguaggio `call`? No. Seguendo le produzioni si arriva a `id ( id , param )` e la derivazione si blocca. Avendo esaurito tutte le possibili alternative di espansione, possiamo affermare con certezza che non appartiene al linguaggio.

Nelle grammatiche context-free, in una singola forma sentenziale possono esserci più non-terminali. Quale espandiamo per primo? Abbiamo due strategie principali:

- **Leftmost derivation** ($\Rightarrow_{lm}$): si sostituisce sempre il non-terminale più a sinistra.
- **Rightmost derivation** ($\Rightarrow_{rm}$): si sostituisce sempre il non-terminale più a destra.

$$
\begin{aligned}
list &\rightarrow list + digit \mid list - digit \mid digit \quad (1)\\
digit &\rightarrow \mathbf{0} \mid \mathbf{1} \mid \cdots \mid \mathbf{9} \quad (2)
\end{aligned}
$$

Derivazione per $9 - 5 + 2$	
1. *list*	
2. *list* + *digit*	1.1
3. *list* − *digit* + *digit*	1.2
4. *digit* − *digit* + *digit*	1.3
5. **9** − *digit* + *digit*	2.10
6. **9** − **5** + *digit*	2.6
7. **9** − **5** + **2**	2.3



Esempio: Derivazione Leftmost e Rightmost

Consideriamo la grammatica per semplici espressioni aritmetiche:

1. $list \rightarrow list + digit \mid list - digit \mid digit$
2. $digit \rightarrow 0 \mid 1 \mid \cdots \mid 9$

Derivazione leftmost per la stringa $9 - 5 + 2$:

1. $list$
2. $\Rightarrow_{lm} list + digit$ (usando $list \rightarrow list + digit$)
3. $\Rightarrow_{lm} list - digit + digit$ (usando $list \rightarrow list - digit$)
4. $\Rightarrow_{lm} digit - digit + digit$ (usando $list \rightarrow digit$)
5. $\Rightarrow_{lm} 9 - digit + digit$ (usando $digit \rightarrow 9$)
6. $\Rightarrow_{lm} 9 - 5 + digit$ (usando $digit \rightarrow 5$)
7. $\Rightarrow_{lm} 9 - 5 + 2$ (usando $digit \rightarrow 2$)

Derivazione rightmost per $9 - 5 + 2$:

1. $list$
2. $\Rightarrow_{rm} list + digit$
3. $\Rightarrow_{rm} list + 2$
4. $\Rightarrow_{rm} list - digit + 2$
5. $\Rightarrow_{rm} list - 5 + 2$
6. $\Rightarrow_{rm} digit - 5 + 2$
7. $\Rightarrow_{rm} 9 - 5 + 2$

3.4 Parse Tree e Parsing

Parse Tree

Un **Parse Tree** (Albero Sintattico) in una grammatica G è un albero radicato che rispetta le seguenti proprietà:

- La radice è etichettata con il simbolo iniziale S.
- I nodi interni sono esclusivamente non-terminali.
- Le foglie sono terminali o stringhe vuote ϵ.
- Se un nodo interno A ha figli $X_1, \ldots, X_n$ (letti da sinistra a destra), deve esistere necessariamente la produzione $A \rightarrow X_1 \ldots X_n \in P$.

Stabilire se una stringa u appartiene a $L(G)$ significa verificarne l'appartenenza tramite il **Parsing**. Sebbene una CFG sia lo strumento ideale per *definire* formalmente un

linguaggio, non è lo strumento ottimale per *verificarne efficientemente* l'appartenenza. Ricorrere alla generazione esaustiva di tutte le stringhe di lunghezza $|u|$ per cercare una corrispondenza sarebbe infatti computazionalmente insostenibile.

3.5 Ambiguità, Associatività e Precedenza

Una grammatica CF G si definisce **ambigua** se esiste almeno una stringa $u \in L(G)$ per la quale è possibile costruire **due Parse Tree differenti** (cioè esistono due derivazioni strutturalmente diverse che producono la stessa stringa finale). Un classico esempio di grammatica ambigua è quella che descrive gli statement Java-like contenenti costrutti del tipo '`if ( E ) stmt else stmt`' (il cosiddetto problema del *dangling-else*).

Associatività

Un operatore è associativo a sinistra (o a destra) se, trovandosi in presenza di più operatori uguali, l'operando condiviso viene legato all'istanza di sinistra (o di destra). Ad esempio, la sottrazione si associa a sinistra ($9 - 5 - 2 \equiv (9 - 5) - 2$), mentre l'assegnamento si associa a destra ($a = b = c \equiv a = (b = c)$). L'associatività va gestita direttamente a livello di grammatica.

Precedenza

Operatori diversi devono avere priorità diverse (es. `id + id * id` deve essere interpretato come `id + (id * id)`). Per gestire le precedenze nella grammatica:

1. Si definisce l'ordine crescente delle precedenze (es. prima $+$, poi $*$, infine le parentesi ()).
2. Per ogni livello, partendo dal maggiore, si crea un nuovo non-terminale ad-hoc (es. E per le somme, T per le moltiplicazioni, F per i fattori isolati) e vi si collegano le relative produzioni.

Una grammatica è detta ambigua se ammette interpretazioni multiple per la stessa stringa.

L'Ambiguità nelle CFG

Una grammatica CFG $G = (N, T, P, S)$ è **ambigua** se esiste una sentenza $u \in L(G)$ per cui esistono due diversi Parse Tree (Alberi di Derivazione Sintattica). Equivalentemente, è ambigua se esiste una stringa u che ammette due diverse derivazioni *leftmost* (o due diverse derivazioni *rightmost*).

Esempio Classico di Ambiguità: $id + id * id$

Sia data la grammatica per le espressioni:

$$E \to E + E \mid E * E \mid (E) \mid id$$

La stringa $id + id * id$ ha due derivazioni leftmost differenti:
Derivazione 1 (precedenza all'addizione): $E \Rightarrow E + E \Rightarrow id + E \Rightarrow id + E * E \Rightarrow id + id * E \Rightarrow id + id * id$
Derivazione 2 (precedenza alla moltiplicazione): $E \Rightarrow E*E \Rightarrow E+E*E \Rightarrow id + E * E \Rightarrow id + id * E \Rightarrow id + id * id$

3.6 Potenza Espressiva: Regular vs Context-Free

I linguaggi regolari sono un sottoinsieme stretto dei linguaggi context-free. Ogni espressione regolare o NFA può essere tradotto in una CFG.

Costruzione: Da NFA a CFG Regolare

Dato un NFA $N = (\Sigma, S, \delta, s_0, F)$, si può costruire una grammatica CFG (che risulterà essere anche regolare) G tale che $L(N) = L(G)$ con il seguente algoritmo:

1. Per ogni stato $i \in S$ creare un non-terminale A_i.
2. Se esiste una transizione $j \in \delta(i, a)$, aggiungere la produzione $A_i \to aA_j$.
3. Se esiste una ϵ-transizione $j \in \delta(i, \epsilon)$, aggiungere la produzione $A_i \to A_j$.
4. Se i è uno stato finale ($i \in F$), aggiungere la produzione $A_i \to \epsilon$.
5. Se $i = s_0$ (stato iniziale dell'NFA), allora A_i diventa il simbolo iniziale di G.

Tuttavia, le CFG hanno dei limiti ben precisi nella descrizione dei linguaggi di programmazione moderni.

Limiti delle CFG nell'Analisi Sintattica

Esistono pattern che le CFG non possono descrivere. Due esempi classici sono:

1. **Dichiarazione prima dell'uso:** Il linguaggio $L_1 = \{wcw \mid w \in (a|b)^*\}$ non è Context-Free, ma Context-Sensitive. Questo significa che una CFG non può forzare che una variabile sia dichiarata prima di essere utilizzata (perché richiede di "ricordare" w e verificarne l'uguaglianza dopo il separatore c). Ecco perché l'analizzatore sintattico accetta semplicemente qualsiasi **id**, demandando questo controllo all'Analisi Semantica (Symbol Table).
2. **Controllo dei parametri:** Il linguaggio $L_2 = \{a^n b^m c^n d^m \mid n, m \geq 1\}$ non è Context-Free. Una CFG non può verificare che il numero di parametri passati a una funzione sia uguale a quello della sua dichiarazione. Pertanto, la sintassi permette di passare una lista generica di argomenti, e il controllo del numero/tipo viene delegato alle fasi successive.

3.7 Ristrutturazione delle Grammatiche

Per poter effettuare l'analisi sintattica in modo automatico (specialmente il Top-Down Parsing), le grammatiche devono possedere certe proprietà. Dobbiamo eliminare le

ambiguità, la ricorsione a sinistra e i prefissi comuni.

3.7.1 Eliminazione dell'Ambiguità: Dangling Else

Consideriamo la classica grammatica ambigua per l'istruzione **if**:

```
stmt -> if expr then stmt
      | if expr then stmt else stmt
      | other
```

Se abbiamo una stringa come `if E1 then if E2 then S1 else S2`, il parser non sa a quale `if` associare l'`else`. Questa ambiguità si elimina ristrutturando la grammatica. L'idea è impedire che un `if-then` senza `else` finisca nel ramo `then` di un `if-then-else`:

```
stmt -> matched_stmt
      | open_stmt

matched_stmt -> if expr then matched_stmt else matched_stmt
              | other

open_stmt -> if expr then stmt
           | if expr then matched_stmt else open_stmt
```

3.7.2 Eliminazione della Ricorsione a Sinistra

Il parsing Top-Down va in loop infinito se incontra una grammatica ricorsiva a sinistra.

Ricorsione a Sinistra: Teoria e Algoritmo

Definizione: Una CFG è ricorsiva a sinistra se $\exists A \in N$ e $\alpha \in (N \cup T)^*$ tale che $A \xrightarrow{+} A\alpha$. Se $A \to A\alpha \in P$, è ricorsione immediata.
Eliminazione della Ricorsione Immediata: Se abbiamo le produzioni: $A \to A\alpha_1 \mid \cdots \mid A\alpha_m \mid \beta_1 \mid \cdots \mid \beta_n$ (dove i β_i non iniziano per A), possiamo trasformarle in una ricorsione a destra introducendo un nuovo non-terminale A':

$$A \to \beta_1 A' \mid \cdots \mid \beta_n A'$$
$$A' \to \alpha_1 A' \mid \cdots \mid \alpha_m A' \mid \epsilon$$

Algoritmo per Eliminazione Generica: Assumendo l'assenza di cicli ($A \xrightarrow{+} A$) e di ϵ-produzioni, per ogni coppia di non-terminali A_i, A_j (con $j < i$), si sostituiscono le occorrenze di A_j nel corpo di A_i con le espansioni correnti di A_j, per poi eliminare la ricorsione immediata su A_i.

Esempio: Eliminazione Ricorsione Generica

Grammatica originale:

$$S \to Aa \mid b$$
$$A \to Ac \mid Sd \mid \epsilon$$

Imponiamo un ordinamento: $A_1 = S, A_2 = A$.

1. $i = 1$ (S): non c'è ricorsione immediata per S.
2. $i = 2, j = 1$ (A valuta S): Sostituiamo S nel corpo di $A \to Sd$. Usiamo $S \to Aa \mid b$ per ottenere le nuove produzioni per A:

$$A \to Ac \mid Aad \mid bd \mid \epsilon$$

3. Ora in A c'è ricorsione immediata a sinistra ($A \to Ac$ e $A \to Aad$). Applichiamo la regola di eliminazione immediata:

$$A \to bdA' \mid A' \text{ (notare che } \epsilon A' \text{ diventa } A')$$
$$A' \to cA' \mid adA' \mid \epsilon$$

3.7.3 Fattorizzazione a Sinistra

Se due produzioni dello stesso non-terminale iniziano con lo stesso prefisso, un parser predittivo non saprà quale scegliere guardando solo i primi token.

Fattorizzazione a Sinistra

Data una produzione $A \to \alpha\beta_1 \mid \alpha\beta_2$, dove α è il prefisso comune più lungo ($\alpha \neq \epsilon$). L'algoritmo trasforma la grammatica ritardando la scelta. Crea un nuovo non-terminale A' e riscrive le produzioni in questo modo:

$$A \to \alpha A'$$
$$A' \to \beta_1 \mid \beta_2$$

Il parser espanderà $A \to \alpha A'$, "consumerà" l'input che corrisponde al prefisso α, e solo quando si troverà a dover espandere A', il carattere di lookahead sarà sufficiente per distinguere se usare β_1 o β_2.

3.8 Algoritmi di Parsing: Bottom-Up e Top-Down

Esistono due macro-categorie di algoritmi di parsing, le cui prestazioni ($O(n)$) dipendono dalla natura della grammatica utilizzata:

- **Bottom-Up Parsing (Grammatiche LR(k)):** Costruisce l'albero dalle foglie verso la radice individuando sottostringhe valide e rimpiazzandole con i relativi non-terminali. È molto potente ma estremamente complesso da scrivere a mano; per questo ci si affida a generatori automatici come YACC.

- **Top-Down Parsing (Grammatiche LL(k)):** Sebbene permettano di descrivere una classe di linguaggi meno ampia rispetto alle LR(k), le grammatiche LL(k) rendono possibile scrivere parser efficienti ($O(n)$) completamente a mano.

3.8.1 Predictive Parsing e Recursive-Descent

Il **Predictive Parsing** è una tecnica Top-Down applicabile in scenari favorevoli, ovvero quando l'osservazione del carattere in input corrente (*lookahead*) è sufficiente a determinare senza ambiguità quale produzione applicare.

Insiemi First e Regole di Parsing

L'insieme First(α) è costituito da tutti i terminali che possono apparire come primo simbolo in una stringa derivabile da α. Il Predictive Parsing è attuabile se e solo se, per ogni coppia di produzioni alternative $A \to \beta$ e $A \to \gamma$, le loro intersezioni sono vuote ($\text{First}(\beta) \cap \text{First}(\gamma) = \emptyset$). Durante il parsing del nodo A con lookahead t: 1. Se esiste un'espansione per cui $t \in \text{First}(\alpha)$, si espande A con α. 2. Altrimenti, se esiste la produzione $A \to \epsilon$, la si applica. 3. Se nessuna condizione è verificata, viene generato un errore di sintassi.

Il **Recursive-Descent** è l'implementazione pratica del Predictive Parsing tramite codice procedurale. Si definisce una funzione ricorsiva per ogni non-terminale che analizza il lookahead e decide quale sotto-procedura invocare. Tuttavia, il Recursive-Descent fallisce di fronte a due difetti grammaticali:

1. **Incrocio degli insiemi First:** Se produzioni diverse iniziano con lo stesso simbolo, il parser non sa quale scegliere. La soluzione è la **Fattorizzazione** (creare un nuovo non-terminale per la porzione di destra divergente).
2. **Ricorsione Sinistra:** Produzioni della forma $A \to A\alpha$ causano loop infiniti. La soluzione consiste nel trasformare la ricorsione sinistra in ricorsione destra ($A \to \beta R$ e $R \to \alpha R \mid \epsilon$).

3.9 Top-Down Parsing: First, Follow e Tabelle LL(1)

L'obiettivo del Top-Down Parsing è trovare una derivazione leftmost costruendo il Parse Tree dalla radice (S) verso le foglie. Se fatto tramite Recursive-Descent senza backtracking (Parser Predittivo), abbiamo bisogno di calcolare due funzioni fondamentali: **First** e **Follow**.

Funzioni First e Follow

Sia $G = (N, T, P, S)$ una CFG e \$ il simbolo speciale di fine file. **First(α)** è l'insieme dei terminali che sono il primo carattere di una stringa derivabile dalla forma sentenziale α:

$$First(\alpha) = \{u \in T \mid \exists \beta \in (N \cup T)^*.\alpha \xrightarrow{*} u\beta\}$$

Se $\alpha \xrightarrow{*} \epsilon$, allora $\epsilon \in First(\alpha)$. **Follow($A$)** è l'insieme dei terminali che si trovano *immediatamente dopo* il non-terminale A in una forma sentenziale derivabile da S:

$$Follow(A) = \{u \in T \mid \exists \alpha, \beta \in (N \cup T)^*.S \xrightarrow{*} \alpha A u \beta\}$$

Nota: $\$ \in Follow(A)$ se A può trovarsi alla fine di una stringa derivata.

Calcolo Pratico di First e Follow

Per calcolare First(X) per un singolo simbolo:

1. Se X è un terminale, $First(X) = \{X\}$.
2. Se $X \to \epsilon \in P$, aggiungi ϵ a $First(X)$.
3. Se $X \to Y_1 Y_2 \dots Y_k \in P$, aggiungi i non-$\epsilon$ di $First(Y_1)$. Se $\epsilon \in First(Y_1)$, aggiungi i non-ϵ di $First(Y_2)$, e così via. Se tutti gli Y_i derivano ϵ, aggiungi ϵ a $First(X)$.

Per calcolare Follow(A):

1. Aggiungi \$ a $Follow(S)$.
2. Per ogni produzione $B \to \alpha A \beta$, aggiungi tutto $First(\beta) \setminus \{\epsilon\}$ a $Follow(A)$.
3. Se $B \to \alpha A$ oppure se $B \to \alpha A \beta$ (dove $\epsilon \in First(\beta)$), tutto ciò che segue B seguirà anche A: quindi aggiungi $Follow(B)$ a $Follow(A)$.

Esempio: First e Follow delle Espressioni Aritmetiche

$$\begin{aligned}
E &\to TE' \\
E' &\to +TE' \mid \epsilon \\
T &\to FT' \\
T' &\to *FT' \mid \epsilon \\
F &\to (E) \mid id
\end{aligned}$$

First: $First(F) = \{(, id\}$; $First(T) = First(F) = \{(, id\}$; $First(E) = First(T) = \{(, id\}$.
$First(E') = \{+, \epsilon\}$; $First(T') = \{*, \epsilon\}$.
Follow: $Follow(E) = \{), \$\}$ (per l'assioma S e per le parentesi).
$Follow(E') = Follow(E) = \{), \$\}$.
$Follow(T) = First(E') \cup Follow(E) = \{+,), \$\}$.
$Follow(T') = Follow(T) = \{+,), \$\}$.
$Follow(F) = First(T') \cup Follow(T) = \{*, +,), \$\}$.

3.9.1 Grammatiche LL(1)

Per poter costruire un parser predittivo deterministico, la grammatica deve appartenere alla classe LL(1) (Lettura da Sinistra, Derivazione Leftmost, 1 carattere di lookahead).

Definizione di Grammatica LL(1)

Una CFG $G = (N, T, P, S)$ è LL(1) se e solo se, per ogni coppia di produzioni alternative $A \to \alpha \mid \beta$:

1. $First(\alpha) \cap First(\beta) = \emptyset$ (i primi caratteri generabili non devono intersecarsi).
2. Se $\epsilon \in First(\beta)$, allora deve essere vero che $First(\alpha) \cap Follow(A) = \emptyset$. Questo significa che se β può sparire, il terminale che incontreremo (che apparterrà a $Follow(A)$) non deve poter essere generato anche da α.
3. Simmetricamente, se $\epsilon \in First(\alpha)$, allora $First(\beta) \cap Follow(A) = \emptyset$.

3.9.2 Predictive Parsing e Costruzione della Tabella M

Un parser predittivo non ricorsivo usa uno Stack (inizializzato con $\$S$), un buffer di input e una Tabella di Parsing bidimensionale $M[A, a]$, dove A è un non-terminale e a è il terminale in input.

Algoritmo di Costruzione della Tabella M

Per ogni produzione $A \to \alpha$ della grammatica, esegui i seguenti passi:

1. Per ogni terminale $a \in First(\alpha)$, inserisci la produzione $A \to \alpha$ nella cella $M[A, a]$.
2. Se $\epsilon \in First(\alpha)$ (ovvero se la produzione può derivare la stringa vuota), per ogni terminale $b \in Follow(A)$, inserisci $A \to \alpha$ nella cella $M[A, b]$.
3. Se $\epsilon \in First(\alpha)$ e anche $\$ \in Follow(A)$, inserisci $A \to \alpha$ nella cella $M[A, \$]$.

Se una cella della tabella si ritrova ad avere **più di una produzione al suo interno**, significa che la grammatica **non è LL(1)**. Questa tabella rappresenta il "cervello" del parser: dice sempre in modo univoco quale produzione espandere guardando lo stack e l'input.

Risoluzione dei conflitti a livello di Parser

A volte l'ambiguità può essere tollerata e risolta manipolando forzatamente la tabella M. Nel caso del "Dangling else", se creiamo la tabella M con la grammatica semplificata (e non con quella ristrutturata che è enorme), troveremo un conflitto nella cella corrispondente all'espansione del ramo opzionale. Se forziamo la tabella scartando l'ϵ-produzione e mantenendo l'accoppiamento con **else**, costringiamo il parser a legare l'`else` all'`if` più vicino, risolvendo l'ambiguità in fase di implementazione pratica.

3.9.3 Meccanica del Predictive Parsing senza ricorsione

Per una CFG LL(1) $G = (N, T, P, S)$ possiamo implementare il Predictive Parsing usando uno stack esplicito invece delle chiamate a funzione ricorsive. Lo stack conterrà il simbolo \$ alla base, seguito da simboli in $(N \cup T)^*$. Ad un generico momento della computazione, se w è l'input visto finora e $\alpha\$$ è il contenuto dello stack, abbiamo che:

$$S \overset{*}{\Rightarrow}_{lm} w\alpha$$

Le componenti operative in gioco durante la simulazione sono:

- w è la stringa in input.
- $*ip$ è il carattere puntato dal puntatore ip (ovvero il nostro simbolo di lookahead corrente nell'input).
- Lo stack viene inizializzato a $S\$$.

4

Traduzione Semantica e Rappresentazioni Intermedie

4.1 Traduzione Guidata dalla Sintassi

Ipotizziamo di operare a valle di un Lexer (l'input è già tokenizzato) e di avere a disposizione un Parser funzionante. Vogliamo tradurre espressioni aritmetiche in **notazione postfissa** (dove l'operatore segue gli operandi, es. E_1E_2 op). Esistono due metodologie principali:

4.1.1 Syntax-Directed Semantic Definitions (Approccio Dichiarativo)

SDSD: Regole e Attributi

Una definizione semantica associa a ogni simbolo della grammatica un set di **attributi** (indicati con $X.p$) e a ogni produzione un set di **regole semantiche** della forma $X.p = f(P)$.

- **Attributi Sintetizzati:** Un attributo è sintetizzato se è definito esclusivamente in funzione degli attributi dei figli (i simboli nel corpo della produzione) o di altri attributi propri della testa. Il loro calcolo è semplice e richiede una singola visita in profondità dell'albero (bottom-up).
- **Attributi Ereditati:** Un attributo è ereditato se dipende dagli attributi dei "fratelli" all'interno del corpo della produzione o dagli attributi del "padre". Il loro calcolo è più complesso e spesso non permette traduzioni "al volo" durante il parsing.

4.1.2 Syntax-Directed Translation Scheme (Approccio Imperativo)

In questo schema, si aggiungono direttamente all'interno del corpo delle produzioni delle **azioni semantiche** (es. $\{print('+')\}$). Queste azioni sono veri e propri frammenti di codice scritti nel linguaggio di programmazione del compilatore. Possono essere visualizzate come "foglie extra" dell'albero sintattico e vengono eseguite in ordine sequenziale durante una visita *Depth-First* del Parse Tree. Questo approccio garantisce la stessa potenza espressiva delle SDSD ma risulta molto più pratico in fase di implementazione.

4.2 Abstract Syntax Tree

L'**Abstract Syntax Tree (AST)** è la struttura ad **albero** più asciutta e minimalista per rappresentare il codice: i **nodi interni** sono esclusivamente gli operatori, mentre **i**

figli (e le foglie) sono gli operandi. La strategia generale di sviluppo (che procede in senso inverso rispetto alle classiche fasi di compilazione) consiste nel:

1. Progettare una grammatica Astratta (G_A), la cui struttura si avvicini il più possibile all'AST.
2. Definire su G_A lo schema di traduzione (Translation Scheme).
3. Trasformare G_A in una grammatica Concreta (G_C) eliminando ricorsione sinistra e ambiguità per renderla idonea al parsing, facendo attenzione a riposizionare correttamente le azioni semantiche affinché la traduzione finale rimanga inalterata.

4.3 Symbol Table per la Gestione dello Scope

La **Symbol Table** è la struttura dati fondamentale usata in tutto il processo.

- **Nel Lexer:** Viene usata per registrare le stringhe lette (es. il token associato a un identificatore con conterrà un puntatore `p` alla sua entry nella tabella).
- **Nel Parser:** Viene sfruttata per gestire ambiti complessi, come il **Lexical Scoping** (Scope Statico).

Regola "Most-closely nested" e Implementazione

In un linguaggio basato su blocchi annidati, un'occorrenza di una variabile x ricade nello scope della dichiarazione presente nel blocco *più interno e vicino* che la racchiude. Per implementare questa logica si utilizzano le **Chained SymTabs** (Tabelle concatenate):

- Ogni blocco crea un nuovo *Environment* locale (gestito tramite tabella hash).
- Questo Environment possiede un riferimento (`prev`) all'Environment genitore.
- Le tabelle vengono istanziate all'ingresso del blocco e distrutte all'uscita, garantendo così che la risoluzione dei nomi risalga la catena in modo corretto.

4.4 Generazione di Rappresentazioni Intermedie (IR)

Esistono due macro-categorie di Rappresentazioni Intermedie:

1. **Ad Albero (Parse Tree, AST):** Potenti, preservano la struttura del codice ad alto livello e sono ideali per l'analisi semantica.
2. **Lineari (es. Three-Address Code):** Perdono l'informazione strutturale del sorgente, ma sono essenziali per le successive fasi di ottimizzazione a basso livello.

Il Three-Address Code (3-AC)

Il 3-AC utilizza un instruction set elementare in cui figurano istruzioni di assegnamento, accesso ad array e salti condizionati (`ifTrue`, `goto`). Le istruzioni ammettono al massimo tre "indirizzi" (tipicamente un target e due operandi, che possono essere variabili, costanti o temporanei generati dal compilatore). Semplificando radicalmente il C (rimuovendo cicli complessi e strutture dati avanzate) si ottiene un set molto simile al 3-AC, talvolta chiamato ∞-Address Code, utile per sviluppare "Piggyback Compiler" che demandano a `gcc` l'ultimo step di compilazione.

4.4.1 Traduzione dall'AST al 3-Address Code: l-value e r-value

Nella traduzione di assegnamenti e accessi ad array, è cruciale distinguere tra il valore e l'indirizzo di memoria associati a un'espressione:

- **l-value (left-value):** Rappresenta una locazione in memoria idonea a comparire a sinistra di un operatore =.
- **r-value (right-value):** Rappresenta un dato concreto, idoneo a comparire alla destra dell'=.

Ogni espressione possiede un *r-value*, ma solo alcune (come variabili o accessi ad array) possiedono anche un *l-value*. Per gestire la traduzione si adottano due funzioni mutuamente ricorsive:

- `lvalue(x: Expr)`: Genera il codice per calcolare l'espressione e restituisce un nuovo nodo rappresentante la *locazione* di memoria risultante. Se deve risolvere un accesso del tipo $y[z]$, valuterà prima l'indice z delegando a `rvalue(z)`.
- `rvalue(x: Expr)`: Affronta quattro scenari principali:
  1. **Foglia:** Restituisce direttamente la costante o l'identificatore, recuperando i dati dalla SymTab.
  2. **Operatore aritmetico/relazionale:** Crea un temporaneo t, invoca ricorsivamente `rvalue` sui due operandi, genera l'istruzione in 3-AC e restituisce t.
  3. **Accesso ad array ($y[z]$):** Semplifica l'indice z con `rvalue(z)`, esegue l'accesso ed emette il risultato su un temporaneo.
  4. **Assegnamento ($y = z$):** Semplifica prima il lato destro valutando $z' =$ `rvalue`(z), poi il lato sinistro con `lvalue(y)`, per poi emettere infine la linea di assegnamento $y = z'$.

Esempio di generazione del codice per `a[i] = 2*a[j-k]`: 1. `t3 = j-k` (valutazione della sottrazione per l'indice interno) 2. `t2 = a[t3]` (accesso all'array interno) 3. `t1 = 2*t2` (moltiplicazione per 2) 4. `a[i] = t1` (assegnamento finale al l-value esterno)